

Generalisation in theory and practice

Christos Dimitrakakis

September 7, 2026

Outline

Classification

The classifier as a decision rule

A decision rule $\pi(a|x)$ generates a **decision** $a \in [m]$. It is the conditional probability of a given x .

Classification

The classifier as a decision rule

A decision rule $\pi(a|x)$ generates a **decision** $a \in [m]$. It is the conditional probability of a given x .

Deterministic predictions given a model $P(y|x)$

Here, we pick the most likely class:

$$\pi(a|x_t) = \mathbb{I} \left\{ a = \arg \max_y P(y|x_t) \right\}$$

Classification

The classifier as a decision rule

A decision rule $\pi(a|x)$ generates a **decision** $a \in [m]$. It is the conditional probability of a given x .

Deterministic predictions given a model $P(y|x)$

Here, we pick the most likely class:

$$\pi(a|x_t) = \mathbb{I} \left\{ a = \arg \max_y P(y|x_t) \right\}$$

Randomised predictions given a model $P(y|x)$

Here, we randomly select a class according to our model:

$$\pi(a|x_t) = P(y_t = a|x_t)$$

Accuracy as a classification metric

Accuracy as a classification metric

The accuracy of a single decision

$$U(a_t, y_t) = \mathbb{I}\{a_t = y_t\} = \begin{cases} 1, & \text{if } a_t = y_t \\ 0, & \text{otherwise} \end{cases}$$

Accuracy as a classification metric

The accuracy of a single decision

$$U(a_t, y_t) = \mathbb{I}\{a_t = y_t\} = \begin{cases} 1, & \text{if } a_t = y_t \\ 0, & \text{otherwise} \end{cases}$$

The accuracy on a dataset

Let $D = \{(x_t, y_t) : t \in [T]\}$ be a dataset. We can measure the accuracy:

$$U(\pi, D) \triangleq \frac{1}{T} \sum_{t=1}^T \pi(y_t | x_t)$$

The expected accuracy of a decision rule

Accuracy as a classification metric

The accuracy of a single decision

$$U(a_t, y_t) = \mathbb{I}\{a_t = y_t\} = \begin{cases} 1, & \text{if } a_t = y_t \\ 0, & \text{otherwise} \end{cases}$$

The accuracy on a dataset

Let $D = \{(x_t, y_t) : t \in [T]\}$ be a dataset. We can measure the accuracy:

$$U(\pi, D) \triangleq \frac{1}{T} \sum_{t=1}^T \pi(y_t | x_t)$$

The expected accuracy of a decision rule

If $(x, y) \sim P$, the accuracy U of a stochastic decision rule π under the distribution P is the probability it predicts correctly

$$U(\pi, P) \triangleq \int_{\mathcal{X}} dP(x) \sum_{y=1}^m P(y|x) \pi(y|x)$$

Beyond classification: Generalised decision rules

Consider a spam application, where the e-mail client can decide between different action for emails. Different actions being best for each type of e-mail. The quality of each action can be captured through a utility function.

Utility of the spam decision problem

What utility function would you use for the spam detection problem?

Utility	Pass	Flag	Trash
Normal			
Spam			
Virus			

The utility function $U: \mathcal{Y} \times \mathcal{A} \rightarrow \mathbb{R}$

The utility function $U(y, a)$ is a real-valued function so that, for a label y , we prefer taking action a to a' iff $U(y, a) > U(y, a')$.

The optimal decision

- ▶ A **model** $P(y|x)$ of class probabilities
- ▶ A **utility** $U(y, a)$ for each class and action combination

The optimal decision

- ▶ A **model** $P(y|x)$ of class probabilities
- ▶ A **utility** $U(y, a)$ for each class and action combination

Expected utility

We can calculate the expected utility of any decision

$$\mathbb{E}[U|a, x] = \sum_y P(y|x, a) U(y, a) = \sum_y P(y|x) U(y, a)$$

Here the first equality follows from the definition of conditional expectation and $P(y|x, a) = P(y|x)$ as the label does not depend on our actions.

The optimal decision

- ▶ A **model** $P(y|x)$ of class probabilities
- ▶ A **utility** $U(y, a)$ for each class and action combination

Expected utility

We can calculate the expected utility of any decision

$$\mathbb{E}[U|a, x] = \sum_y P(y|x, a) U(y, a) = \sum_y P(y|x) U(y, a)$$

Here the first equality follows from the definition of conditional expectation and $P(y|x, a) = P(y|x)$ as the label does not depend on our actions.

The optimal decision

For any observation x , and P , we take the action maximising expected utility:

$$a^* = \arg \max_a \mathbb{E}_P[U|a, x]$$

This defines a function $\mathcal{X} \rightarrow \mathcal{A}$, which is the Bayes-optimal decision rule.

The optimal decision rule

- ▶ A **model** $P(y|x)$ of class probabilities
- ▶ A **utility** $U(y, a)$ for each class and action combination
- ▶ A **decision rule** $\pi(a|x)$ assigning probability to action a for every possible input x

The optimal decision rule

- ▶ A **model** $P(y|x)$ of class probabilities
- ▶ A **utility** $U(y, a)$ for each class and action combination
- ▶ A **decision rule** $\pi(a|x)$ assigning probability to action a for every possible input x

Expected utility over a dataset.

We obtain the expected utility of the **decision rule** by marginalising over all actions

$$U(\pi, D) \triangleq \mathbb{E}[U|\pi, D] \stackrel{D=(x_t, y_t)_{t=1}^T}{=} \sum_{t=1}^T \mathbb{E}[U|\pi, x_t] = \sum_{t=1}^T \sum_{a \in \mathcal{A}} U(y_t, a) \pi(a|x_t)$$

Here the first equality follows from the definition of conditional expectation and $P(y|x, a) = P(y|x)$ as the label does not depend on our actions.

The optimal decision rule

- ▶ A **model** $P(y|x)$ of class probabilities
- ▶ A **utility** $U(y, a)$ for each class and action combination
- ▶ A **decision rule** $\pi(a|x)$ assigning probability to action a for every possible input x

Expected utility over a dataset.

We obtain the expected utility of the **decision rule** by marginalising over all actions

$$U(\pi, D) \triangleq \mathbb{E}[U|\pi, D] \stackrel{D=(x_t, y_t)_{t=1}^T}{=} \sum_{t=1}^T \mathbb{E}[U|\pi, x_t] = \sum_{t=1}^T \sum_{a \in \mathcal{A}} U(y_t, a) \pi(a|x_t)$$

Here the first equality follows from the definition of conditional expectation and $P(y|x, a) = P(y|x)$ as the label does not depend on our actions.

Expected utility over P

We can marginalise over possible datasets D

$$U(\pi, P) \triangleq \mathbb{E}_P^\pi[U] = \int_{\mathcal{D}} dP(D) U(\pi, D) \stackrel{\text{i.i.d.}}{=} \int_{\mathcal{X}} dP(x) \sum_y P(y|x) \sum_a \pi(a|x) U(y, a)$$

Taking into account the probability

- ▶ For classification, it makes sense to look at the probability of the labels.
- ▶ If we are not very confident about our prediction, this should be taken into account:
- ▶ Define $P(y|x)$ to be our classifier's probability for label y , given features x . Then we can use two simple metrics:

Precision

The average probability of the actual class:

$$\sum_{t=1}^T P(y_t|x_t) / T$$

- ▶ If we always assign probability 1 to the correct label, this score is 1.
- ▶ If we always assign probability $1/m$ to all labels, the score is $1/m$.

Negative Log-Loss

Here we assign look at the **logarithm** of the probability. This really penalises bad guesses.

- ▶ $\sum_{t=1}^T \ln P(y_t|x_t)/T$
- ▶ If we always assign probability 1 to the correct label, this score is 0.
- ▶ If we assign probability 0 to even a single label, the score is $-\infty$.

```
from sklearn.metrics import log_loss
```

in scikitlearn implements log-loss (**not** negative)

Regression

The regressor as a deterministic decision rule

A decision rule π generates a **decision** $a \in \mathbb{R}^m$.

- ▶ For **deterministic** rules $\pi(x)$ is the prediction for x .
- ▶ We need to define the quality of our predictions, either as a utility $U(y_t, a_t)$ (to maximise) or a loss function $\ell(y_t, a_t)$ (to minimise).

Mean-Squared Error Loss on a Dataset

This is the squared difference in predicted versus actual values:

$$\frac{1}{T} \sum_{t=1}^T [y_t - \pi(x_t)]^2$$

Expected MSE

If $(x, y) \sim P$, the expected MSE of a deterministic rule $\pi : \mathcal{X} \rightarrow \mathbb{R}$ is:

$$\mathbb{E}(y - \pi)^2 = \int_{\mathcal{X}} \mathbb{E}[(y - \pi(x))^2 \mid x] dP(x) = \int_{\mathcal{X}} \int_{\mathcal{Y}} dP(x, y) [y - \pi(x)]^2.$$

Probabilistic regression

The regressor as a stochastic decision rule

A decision rule π generates a **decision** $a \in \mathbb{R}^m$.

- ▶ For **stochastic** rules $\pi(a|x)$ defines a density over predictions.
- ▶ In this case it is natural to define $\pi(y_t, x_t)$ as our metric.

Likelihood on a Dataset

The mean-square error is simply the squared difference in predicted versus actual values:

$$\prod_{t=1}^T \pi(y_t|x_t)$$

We will later see a link between this metric, mean-square error and estimation.

Training and overfitting

Training data

- ▶ $D = ((x_t, y_t) : t = 1, \dots, T)$.
- ▶ $x_t \in \mathcal{X}, y_t \in \mathcal{Y}$.

Training and overfitting

Training data

- ▶ $D = ((x_t, y_t) : t = 1, \dots, T)$.
- ▶ $x_t \in \mathcal{X}, y_t \in \mathcal{Y}$.

Assumption: The data is generated i.i.d.

- ▶ $(x_t, y_t) \sim P$ for all t (identical)
- ▶ $D \sim P^T$ (independent)

Training and overfitting

Training data

- ▶ $D = ((x_t, y_t) : t = 1, \dots, T)$.
- ▶ $x_t \in \mathcal{X}, y_t \in \mathcal{Y}$.

Assumption: The data is generated i.i.d.

- ▶ $(x_t, y_t) \sim P$ for all t (identical)
- ▶ $D \sim P^T$ (independent)

The optimal decision rule for P

$$\max_{\pi} U(\pi, P) = \max_{\pi} \int_{\mathcal{X} \times \mathcal{Y}} dP(x, y) \sum_a \pi(a|x) U(a, y).$$

Here we measure performance for the data **distribution P** .

Training and overfitting

Training data

- ▶ $D = ((x_t, y_t) : t = 1, \dots, T)$.
- ▶ $x_t \in \mathcal{X}, y_t \in \mathcal{Y}$.

Assumption: The data is generated i.i.d.

- ▶ $(x_t, y_t) \sim P$ for all t (identical)
- ▶ $D \sim P^T$ (independent)

The optimal decision rule for P

$$\max_{\pi} U(\pi, P) = \max_{\pi} \int_{\mathcal{X} \times \mathcal{Y}} dP(x, y) \sum_a \pi(a|x) U(a, y).$$

Here we measure performance for the data **distribution P** .

The optimal decision rule for D

$$\max_{\pi} U(\pi, D) = \max_{\pi} \sum_{(x,y) \in D} \sum_a \pi(a|x) U(a, y).$$

Performance is measured over the **data D**

Generalisation

The fundamental problem

- ▶ We want to maximise $U(\pi, P)$, the **actual** performance.
- ▶ We can only measure $U(\pi, D)$, over **some data**.
- ▶ We have a **learning algorithm** $\lambda : \mathcal{D} \rightarrow \Pi$
- ▶ If $\pi = \lambda(D)$, then we instead measure $U(\lambda(D), D)$.
- ▶ However $\mathbb{E}_P[U(\lambda(D), D)] \geq U(\pi, P) \ \forall \pi$ (**biased** estimator)

Generalisation

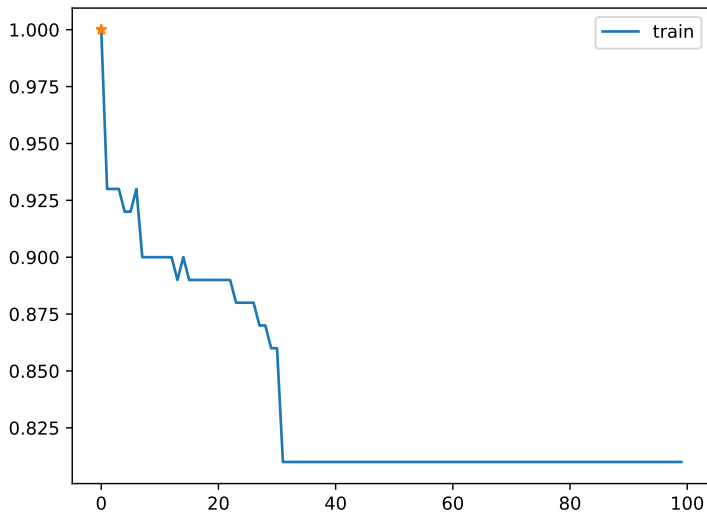
The fundamental problem

- ▶ We want to maximise $U(\pi, P)$, the **actual** performance.
- ▶ We can only measure $U(\pi, D)$, over **some data**.
- ▶ We have a **learning algorithm** $\lambda : \mathcal{D} \rightarrow \Pi$
- ▶ If $\pi = \lambda(D)$, then we instead measure $U(\lambda(D), D)$.
- ▶ However $\mathbb{E}_P[U(\lambda(D), D)] \geq U(\pi, P) \ \forall \pi$ (**biased** estimator)

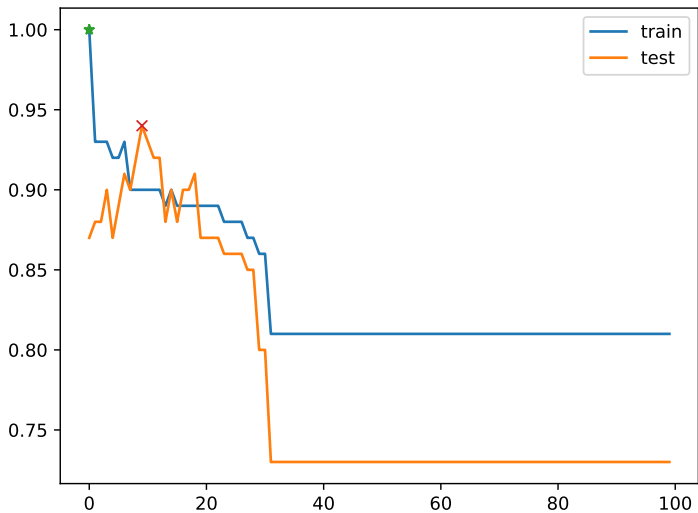
Training and testing

- ▶ Split D in $D_{\text{train}}, D_{\text{test}}$
- ▶ Obtain $\pi = \lambda(D_{\text{train}})$
- ▶ Calculate $U(\pi, D_{\text{test}})$
- ▶ $\mathbb{E}_P[U(\pi, D_{\text{test}})] = U(\pi, P)$ for any fixed π (unbiased estimator)
- ▶ $\mathbb{E}_P[U(\lambda(D_{\text{train}}), D_{\text{train}})] \geq U(\pi, P)$ (biased estimator)

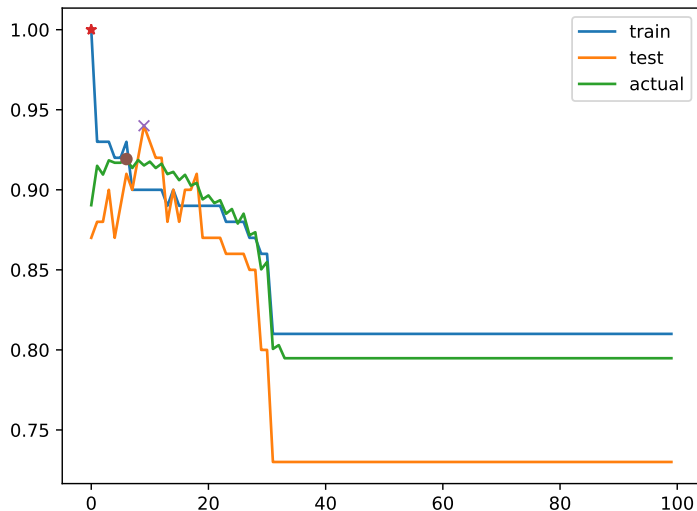
kNN Classifier Accuracy on a single dataset



kNN Classifier Accuracy on a single dataset

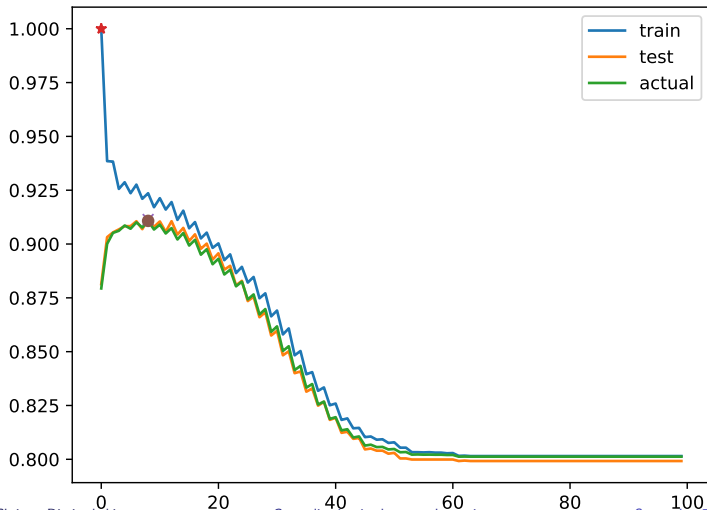


kNN Classifier Accuracy on a single dataset



Expected kNN Classifier Accuracy

Expectation approximated over 100 datasets D sampled from P .



The Train/Validation/Test methodology

Main idea

Use each piece of data once to make decisions and measure

Training set

Use to decide low-level model parameters

Validation set

Use to decide between:

- ▶ different hyperparameters (e.g. K in nearest neighbours)
- ▶ model (e.g. neural networks versus kNN)

Test set

Use to measure the final quality of a model

Cross-validation (XV)

Idea

- ▶ Use XV to select hyperparameters instead of a single train/valid test.

Methodology

- ▶ Split training set D in k different subsets
- ▶ At iteration i
- ▶ Use the i -th subset for validation
- ▶ Use all the remaining $k - 1$ subsets for training
- ▶ Average results on validation sets

Cross-validation example: A mean estimate

Live coding in Python:

- ▶ Get a mean estimate
- ▶ Perform cross-validation

Hand-crafted

```
def xv_shuffler(x, estimator, scoring, n_folds):
    ^Irng = np.random.default_rng
    ^I# shuffle data
    ^IT = len(x)
    ^Iindices = np.arange(T)
    ^Ishuffle(indices)
    ^Ifold_size = np.ceil(T / n_folds)
    ^Ifold_start = np.zeros(n_folds)
    ^Ifold_end = np.zeros(n_folds)
    ^I# create folds
    ^Ifor k in range(n_folds):
    ^I^Ifold_start = k * fold_size
    ^I^Ifold_end = np.min((k+1) * fold_size, T)
    ^I# for each fold:
    ^I# 1. Run estimator on k+1 folds
```

Bootstrapping

- ▶ Express uncertainty by resampling the data.
- ▶ Repeat your calculations for each resample

Bootstrapping

- ▶ Express uncertainty by resampling the data.
- ▶ Repeat your calculations for each resample

BootstrapSample(D)

```
input Data  $D = (z_1, \dots, z_T)$ , of size  $T$   
for  $t \in \{1, \dots, T\}$  do  
    Select  $i$  uniformly in  $[T]$   
    Add the  $i$ -th point to  $D_b$   
end for  
return  $D_b$ 
```

Bootstrapping

- ▶ Express uncertainty by resampling the data.
- ▶ Repeat your calculations for each resample

BootstrapSample(D)

```
input Data  $D = (z_1, \dots, z_T)$ , of size  $T$   
for  $t \in \{1, \dots, T\}$  do  
    Select  $i$  uniformly in  $[T]$   
    Add the  $i$ -th point to  $D_b$   
end for  
return  $D_b$ 
```

BootstrapEstimate(D, λ, N)

```
input Data  $D \in \mathcal{D}$ , algorithm  $\lambda : \mathcal{D} \rightarrow \Theta$ ,  $N > 0$  number of samples  
for  $n \in \{1, \dots, N\}$  do  
     $\theta_n = \lambda(\text{BootstrapSample}(D))$   
end for  
return  $\{\theta_n : n \in [N]\}$ 
```

The wrong way to do XV for subset selection

1. Screen the predictors: find a subset of “good” predictors that show fairly strong (univariate) correlation with the class labels.
2. Using just this subset of predictors, build a multivariate classifier.
3. Use cross-validation to estimate the unknown tuning parameters and to estimate the prediction error of the final model.

Is this a correct application of cross-validation?

Consider a scenario with $N = 50$ samples in two equal-sized classes, and $p = 5000$ quantitative predictors (standard Gaussian) that are independent of the class labels. The true (test) error rate of any classifier is 50%.

The right way to do XV for feature selection

1. Divide the samples into K cross-validation folds (groups) at random.
2. For each fold $k = 1, 2, \dots, K$
3. Find a subset of “good” predictors that show fairly strong (univariate) correlation with the class labels, using all of the samples except those in fold k .
4. Using just this subset of predictors, build a multivariate classifier, using all of the samples except those in fold k .
5. Use the classifier to predict the class labels for the samples in fold k .

Generalisation in theory and practice

└ Estimating quality

└ Methodology

└ The right way to do XV for feature selection

The right way to do XV for feature selection

1. Divide the samples into K cross-validation folds (groups) at random.
2. For each fold $k = 1, 2, \dots, K$.
3. Find a subset of "good" predictors that show fairly strong (univariate) correlation with the class labels, using all of the samples except those in fold k .
4. Using just this subset of predictors, build a multivariate classifier, using all of the samples except those in fold k .
5. Use the classifier to predict the class labels for the samples in fold k .

Level notes

- **BSc:** The point isn't the procedure itself: it's noticing that cross-validation can silently give you a fake guarantee if information from outside a fold leaked into the model-building step before validation happened. Spotting this kind of methodological trap is a core scientific-method skill, and one you should actively check for in your own project.
- **MSc:** **Why** does the wrong version give a near-zero apparent error despite a true error of 50%? Connect it to the estimation-error bound above: what is $|\hat{\Pi}|$ effectively doing once you've screened 5000 predictors first?

Lab

- ▶ scikitlearn KNN
- ▶ scikitlearn performance measures
- ▶ train/test/validate plot with increasing k
- ▶ XV plot with increasing k
- ▶ Scaling and other preprocessing
- ▶ Effect of outliers on preprocessing
- ▶ Optional: Bootstrap performance evaluation

Learning and generalisation

How well can decision rule perform?

Estimation theory view

- ▶ Bias: The **expected difference** between the estimated value and the unknown parameter
- ▶ Variance: Just the **variance** of that estimator, as a random variable.
- ▶ Related to generalisation error.

Learning theory view

- ▶ Approximation error: How well a class of rules can approximate the optimal one.
- ▶ Estimation error: How easy it is to choose the best rule in the class.

Which one do we use?

This course mainly follows the **Learning theory view**: it talks directly about $U(\pi, P)$, the quantity we actually care about, rather than about estimating some unrelated parameter θ . The **Estimation theory view** is covered next for completeness. It's the classical framing you'll meet in statistics courses, and the two turn out to be closely related, but it doesn't add much that's new about

Unbiased estimators

Definition (Estimator)

An estimator is a function $f: \mathcal{D} \rightarrow \Theta$, where Θ is a set of parameters. For any given dataset $D \in \mathcal{D}$, it returns a single estimate $\hat{\theta} = f(D)$.

Definition (Unbiased estimator)

An estimator is **unbiased** if, for any θ and corresponding distribution $P(D|\theta)$:

$$\mathbb{E}[f | \theta] = \sum_D f(D)P(D|\theta) = \theta.$$

Example (Sample mean estimator)

Consider $D = (x_1, \dots, x_T)$ with $x_t \sim P$ being i.i.d samples with $\mathbb{E}[x_t] = \theta$. The sample mean estimator $f(D) = \sum_t x_t / T$ is unbiased, as :

$$\mathbb{E}[f] = \mathbb{E}\left[\sum_{t=1}^T x_t / T\right] = \frac{1}{T} \sum_{t=1}^T \mathbb{E}[x_t] = \frac{1}{T} \sum_{t=1}^T \theta = \theta.$$